ASSIGNMENT 1

Heart Disease Risk Prediction

An End-to-End MLOps Pipeline

From data exploration and model training to experiment tracking, CI/CD, Docker, Kubernetes deployment and monitoring.

Submitted by	**PownKumar A**
Student ID	2025AE05941
Course	AIMLCZG523 - Machine Learning Operations
GitHub repository	https://github.com/pownkumar-a/heart-mlops
Video recording	https://drive.google.com/drive/folders/1LG2Mi2sufPHRMZ40bPn73uhJcZ_HcPEJ

1. Introduction

Heart disease is one of the main causes of death in the world, and finding high-risk patients early helps doctors to act in time. In this assignment my aim was not only to train a classifier, but also to build the full **MLOps lifecycle** around it: data processing that can be repeated, experiment tracking, automated testing, CI/CD, a Docker container, deployment on Kubernetes and monitoring.

I used the **UCI Heart Disease (Cleveland)** dataset. My model predicts whether a patient has heart disease from 13 clinical features such as age, sex, chest pain type, blood pressure, cholesterol and maximum heart rate. The final model is available as a REST API (`POST /predict`) that returns the prediction and a confidence score.

1.1 Tools used

Table 1: *Technology stack*

Area	Tool / library
Language and notebooks	Python 3.9, Jupyter Notebook
Data and EDA	pandas, matplotlib, seaborn
Modelling	scikit-learn (Pipeline, ColumnTransformer, GridSearchCV), joblib
Experiment tracking	MLflow 2.22 (local file store `mlruns/`)
Testing and CI/CD	pytest, flake8, GitHub Actions
Serving	FastAPI, uvicorn, pydantic
Containers and deployment	Docker 28, minikube (Kubernetes v1.32), kubectl
Monitoring	prometheus-client, Prometheus 2.53, Grafana 11.1

2. System Architecture

Figure 1 shows the overall flow of my project. First I download and explore the data. Then the training script builds a preprocessing + model pipeline, tunes it with cross-validation and logs every run to MLflow. The best pipeline is saved as `models/model.joblib`. Every time I push code to GitHub, a CI workflow runs lint, unit tests and training. Finally, the model is packaged in a Docker image with a FastAPI service, deployed on Kubernetes with two replicas, and monitored with Prometheus and Grafana.

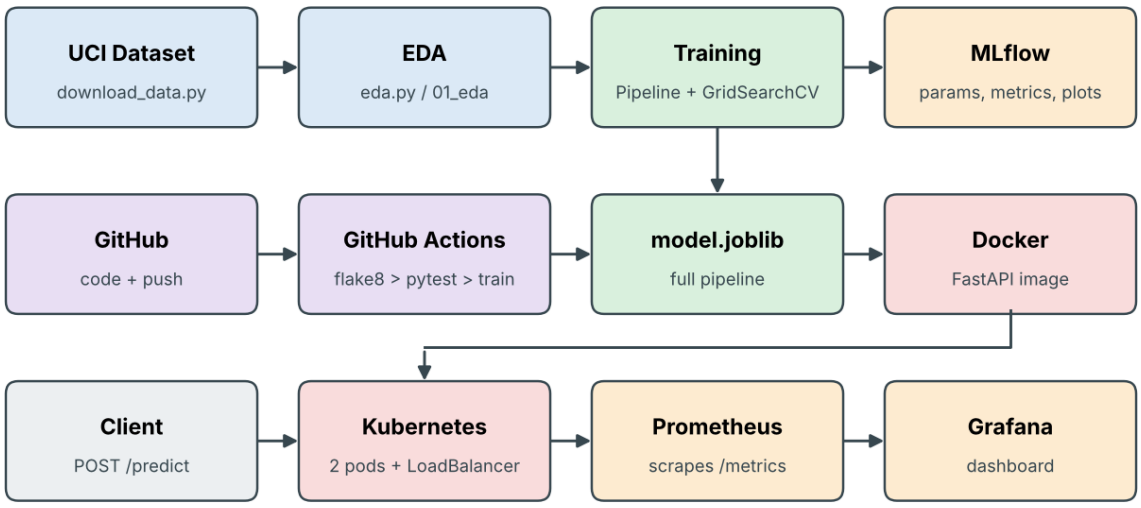


Figure 1: *End-to-end MLOps architecture*

3. Data Acquisition and Exploratory Data Analysis

3.1 Data acquisition

I wrote `src/download_data.py` to download the dataset directly from the UCI Machine Learning Repository (`processed.cleveland.data`). In the original file, missing values are written as `?`, so I read them as `NaN` and added the column names. The file is saved as `data/raw/heart.csv` (303 rows x 14 columns), and a cleaned copy with the missing values filled is saved as `data/processed/heart_clean.csv`. The target column `num` (0-4) was converted into a binary label: **0 = no disease, 1 = disease (num > 0)**.

```
[2]: df.info()
     print("\nMissing values:\n", df.isna().sum()[df.isna().sum() > 0])
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 15 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    float64
 1   sex       303 non-null    float64
 2   cp        303 non-null    float64
 3   trestbps  303 non-null    float64
 4   chol      303 non-null    float64
 5   fbs       303 non-null    float64
 6   restecg   303 non-null    float64
 7   thalach   303 non-null    float64
 8   exang     303 non-null    float64
 9   oldpeak   303 non-null    float64
 10  slope     303 non-null    float64
 11  ca        299 non-null    float64
 12  thal      301 non-null    float64
 13  num       303 non-null    int64
 14  target    303 non-null    int64
dtypes: float64(13), int64(2)
memory usage: 35.6 KB

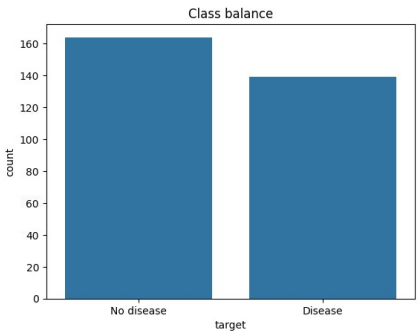
Missing values:
 ca      4
thal     2
dtype: int64
```

Figure 2: *Dataset structure and missing values (ca: 4, thal: 2)*

3.2 Class balance and feature distributions

The two classes are almost balanced: **164 patients without disease (54%) and 139 with disease (46%)**. Because of this I did not need any resampling, but I used stratified splits so that the train and test sets keep the same ratio.

```
[3]: df["target"].value_counts()
```




```
target
0    164
1    139
Name: count, dtype: int64
```

Figure 3: *Class balance of the target*

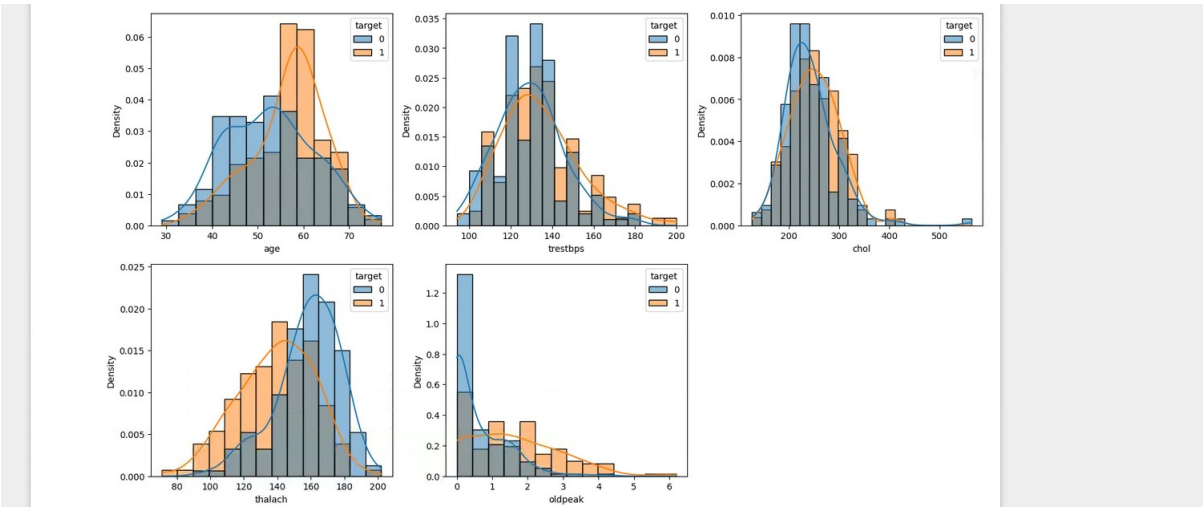


Figure 4: *Distribution of numeric features by target (0 = no disease, 1 = disease)*

From the histograms I observed that patients with disease are older on average (peak around 58-60 years), reach a **lower maximum heart rate** (`thalach`) and have **higher ST depression** (`oldpeak`). Cholesterol and resting blood

pressure look almost the same in both groups. One cholesterol value (about 564 mg/dl) is an outlier, but it is a possible medical value, so I kept it; scaling reduces its effect.

3.3 Correlation analysis

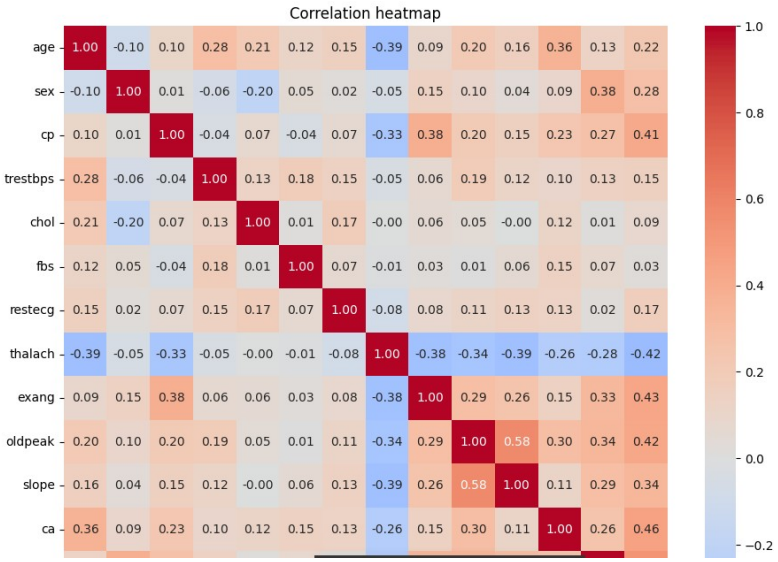


Figure 5: *Correlation heatmap of all features and the target*

The heatmap shows that the features most related to disease are `thal` (0.53), `ca` (0.46), `exang` (0.43), `oldpeak` (0.42) and `cp` (0.41). `thalach` has a strong negative correlation (-0.42). `chol` (0.09) and `fbs` (0.03) are weak predictors. No two features are so strongly correlated that I had to remove one (the highest is oldpeak vs slope, 0.58).

4. Feature Engineering and Model Development

4.1 Preprocessing pipeline

I put all the preprocessing inside a scikit-learn `ColumnTransformer`, which is part of the model `Pipeline`. Because of this, the transformations are learned only from the training folds (no data leakage), and they are saved together with the model, so the API can take raw patient values.

Table 2: *Preprocessing per feature group*

Feature group	Columns	Transformation
Numeric	age, trestbps, chol, thalach, oldpeak, ca	Median imputation + StandardScaler
Binary	sex, fbs, exang	Most-frequent imputation
Categorical codes	cp, restecg, slope, thal	Most-frequent imputation + One-Hot Encoding

4.2 Models and tuning

I split the data into **80% training (242 rows) and 20% test (61 rows)** with stratification and a fixed seed (42). I tuned two models with `GridSearchCV` using **5-fold stratified cross-validation** and ROC-AUC as the score:

- **Logistic Regression** - C in {0.01, 0.03, 0.1, 0.3, 1, 3, 10} (7 combinations).
- **Random Forest** - n_estimators {100, 300}, max_depth {3, 5, None}, min_samples_leaf {1, 5} (12 combinations).

For the best settings of each model, I calculated accuracy, precision, recall, F1 and ROC-AUC with cross-validation (mean +/- std), and then once on the test set.

4.3 Results and model selection

Table 3: *Model comparison (LR best C = 0.3; RF best: 300 trees, max_depth 5, min_samples_leaf 5)*

Metric	LR - CV	LR - Test	RF - CV	RF - Test
Accuracy	0.847 +/- 0.022	0.885	0.835 +/- 0.030	0.902
Precision	0.879 +/- 0.071	0.839	0.844 +/- 0.050	0.867
Recall	0.783 +/- 0.062	0.929	0.792 +/- 0.080	0.929
F1-score	0.824 +/- 0.025	0.881	0.813 +/- 0.039	0.897
ROC-AUC	**0.909 +/- 0.018**	0.959	0.901 +/- 0.025	0.960

I selected Logistic Regression because it has the higher cross-validated ROC-AUC (0.909 vs 0.901) and a smaller standard deviation across the folds, which means it is more stable. It is also simpler and easier to explain. I used the test set only for reporting, not for choosing the model. On the test set, my model correctly classifies 54 of the 61 patients and misses only 2 of the 28 disease cases (recall 0.929). This is important for a medical use case, where missing a sick patient is the worst error.

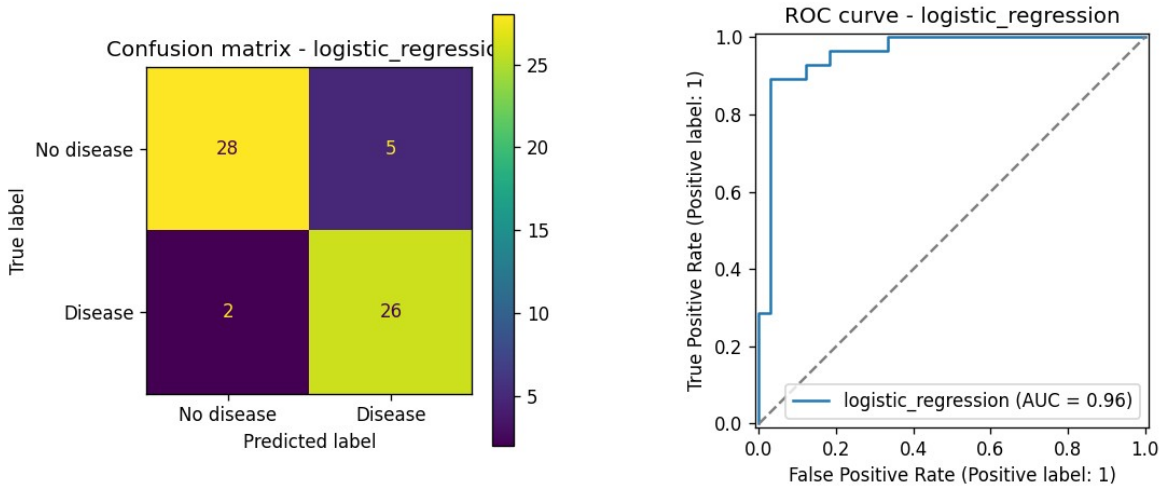


Figure 6: *Confusion matrix and ROC curve of the selected Logistic Regression model on the test set*

5. Experiment Tracking with MLflow

In `src/train.py` I log every experiment to MLflow (experiment name `heart-disease`, stored in `mlruns/`). For each model there is a parent run with the model type, best hyper-parameters, the CV mean and std of all five metrics, the test metrics, the confusion matrix and ROC plots, and the trained model with an input example. I also logged every grid-search combination as a **nested child run** with its parameters and CV ROC-AUC, so all 19 configurations I tried can be compared in the MLflow UI (`mlflow ui --port 5000`).

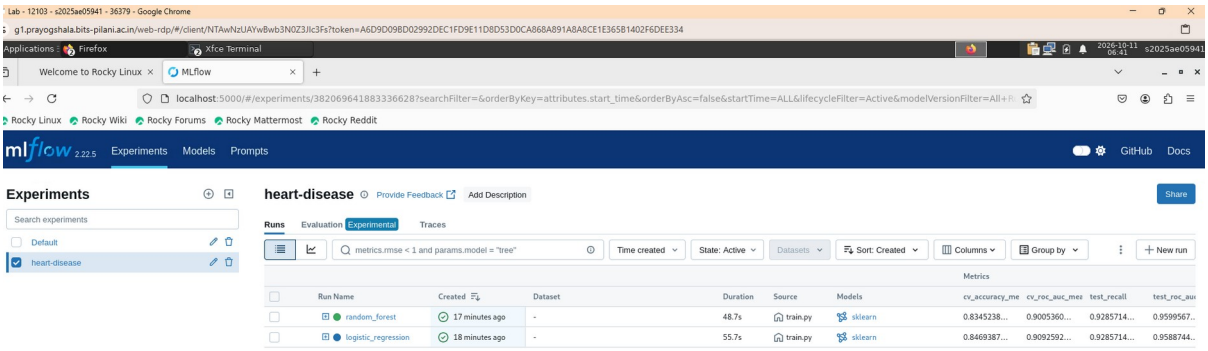


Figure 7: *MLflow experiment with the two parent runs and their logged metrics*

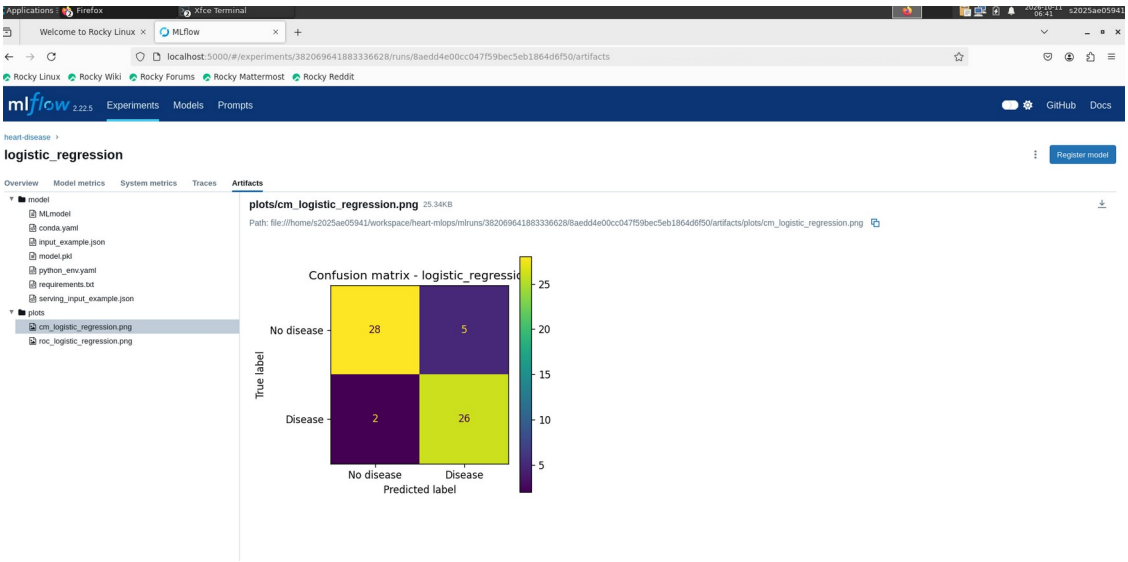


Figure 8: *Artifacts of the Logistic Regression run (model files and plots)*

6. Model Packaging and Reproducibility

- I saved the complete pipeline (preprocessing + model) with joblib as `models/model.joblib`, and all metrics and best parameters in `models/metrics.json`. MLflow also keeps a copy of the model in MLflow format (`MLmodel`, `conda.yaml`, `requirements.txt`).
- I pinned all library versions in `requirements.txt` (training) and `requirements-api.txt` (serving) and used a fixed random seed, so running the training again gives exactly the same results. The CI run on GitHub also produced the same numbers.
- `src/predict.py` loads the model and returns the prediction, label, confidence and disease probability for a list of patients. Missing `ca/thal` values are filled by the imputers inside the pipeline.

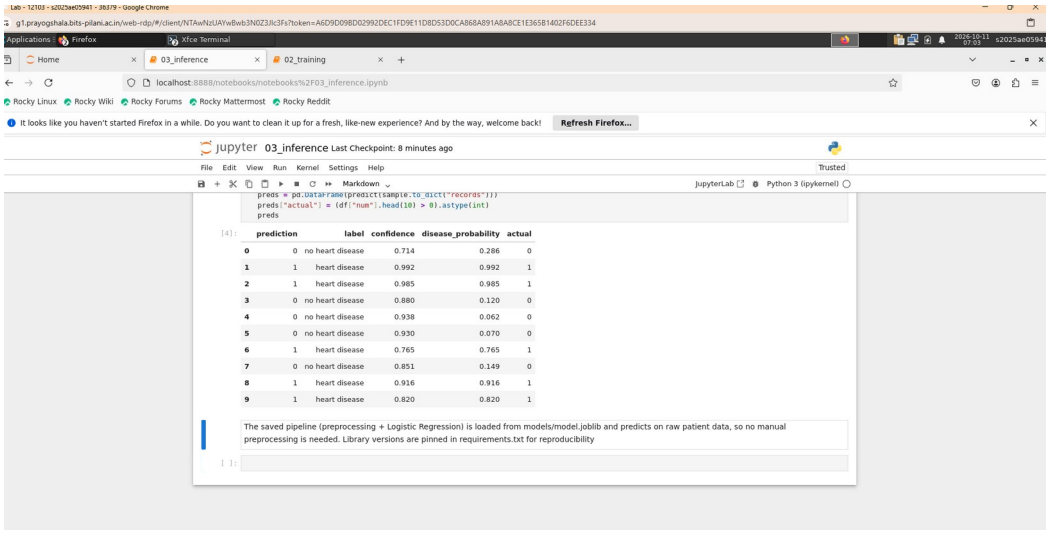


Figure 9: *Inference with the saved model: predictions for the first 10 patients match the actual labels*

7. CI/CD Pipeline and Automated Testing

7.1 Unit tests

I wrote nine unit tests with pytest in the `tests/` folder (run with `pytest -v`), and I check the code style with flake8 (max line length 100).

Table 4: *Unit tests*

Test file	What is tested
test_data.py (3)	Dataset shape (303 x 14), features and binary target, missing values only in ca/thal
test_pipeline.py (3)	Preprocessor removes all NaNs and one-hot encodes; pipeline trains with ROC-AUC > 0.8; both models defined
test_predict.py (3)	Output format and value ranges, missing-value handling, batch prediction

7.2 GitHub Actions workflow

My workflow `.github/workflows/ci.yml` runs on every push and pull request to `main`. Job 1 (**Lint and unit tests**) installs the dependencies, runs flake8 and pytest and uploads the JUnit test report. Job 2 (**Train model**) runs only if job 1 passes; it trains both models and uploads the model, plots, training log and MLflow runs as artifacts.

The pipeline stops when there is an error. My first run failed because flake8 reported `W292 no newline at end of file` and the tests folder was still empty. After I fixed these, all the next runs passed.

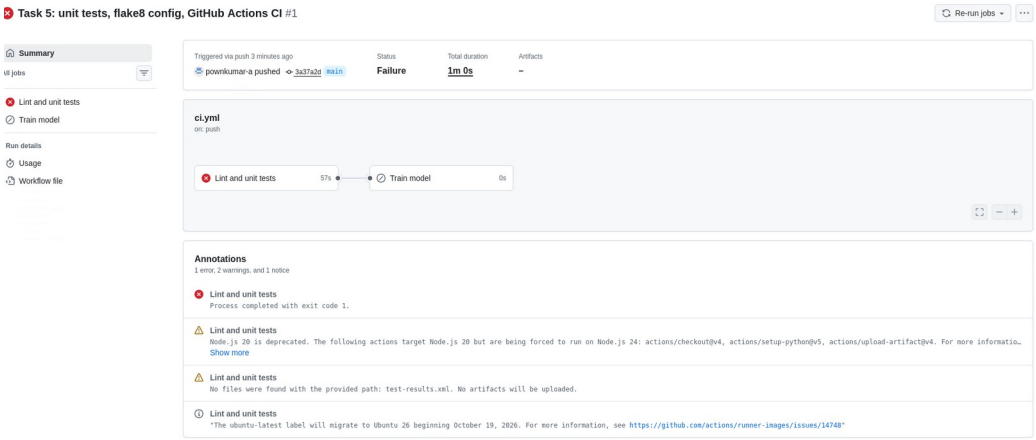


Figure 10: *First CI run failing on lint errors (the training job is skipped)*

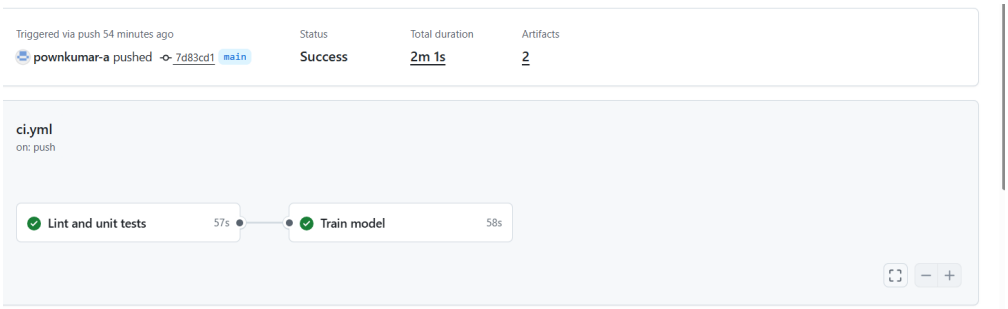


Figure 11: *Successful CI run: lint and tests, then training*

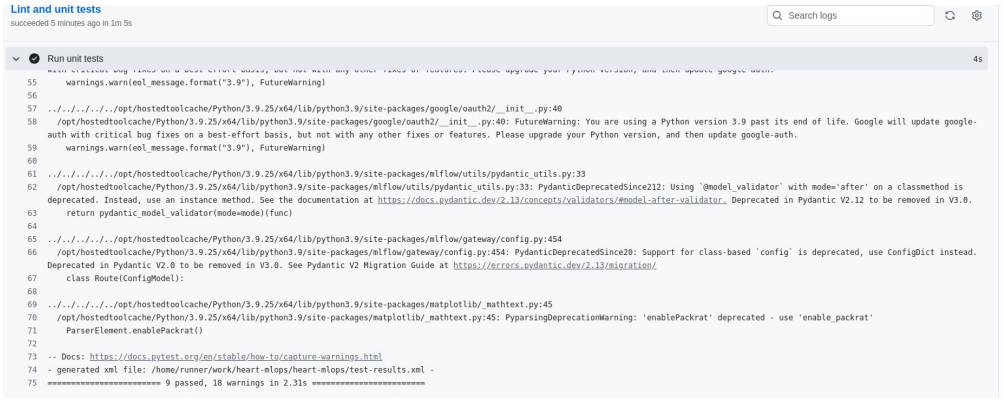


Figure 12: *CI log: all 9 unit tests passed*



Figure 13: *CI log: model training output (same results as the local run)*

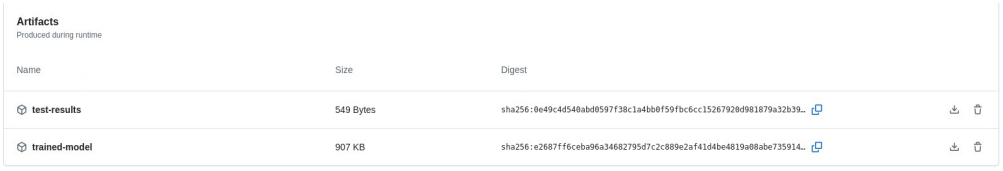


Figure 14: *Artifacts uploaded by the CI run*

8. Model Containerisation

I wrote the API with **FastAPI** (`app/main.py`). It loads the model once when it starts and has the endpoints `GET /health`, `POST /predict`, `GET /metrics` and the Swagger UI at `/docs`. The request body is checked with a pydantic model, so invalid input gets an HTTP 422 error.

My `Dockerfile` uses the `python:3.9-slim` base image from the AWS public registry mirror (because Docker Hub gave a rate-limit error), installs only the serving libraries from `requirements-api.txt`, copies `src/`, `app/` and `models/`, and starts uvicorn on port 8000. A `.dockerignore` file keeps the image small.

```
docker build -t heart-api:1.0 .
docker run -d --name heart-api -p 8000:8000 heart-api:1.0
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" -d '{...}'
```



Figure 15: *Docker image build*

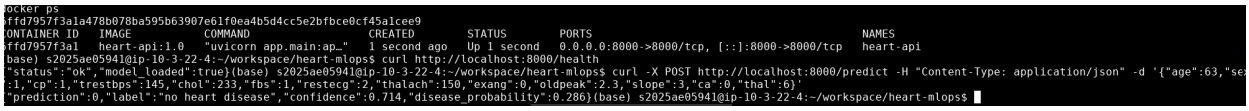


Figure 16: *Running container, health check and a prediction with curl*

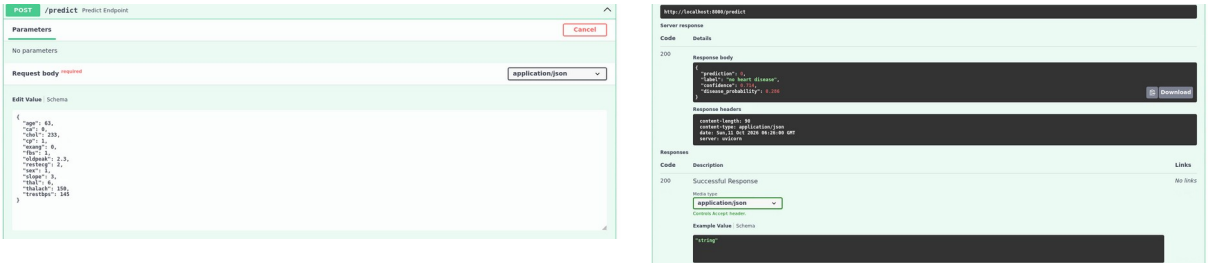


Figure 17: *Swagger UI: /predict request and response (prediction 0, confidence 0.714)*

9. Production Deployment on Kubernetes

I deployed the container on a local **minikube** cluster. I loaded the image into the cluster with `minikube image load heart-api:1.0` and applied the manifests in `k8s/` with `kubectl apply -f k8s/`.

- **deployment.yaml** - 2 replicas, CPU/memory requests and limits, a readiness probe and a liveness probe on `/health`, and Prometheus scrape annotations.
- **service.yaml** - a `LoadBalancer` Service (port 80 to container port 8000, NodePort 30080).

Both pods reached the Running state, and I accessed the API through `minikube service heart-api --url`. On minikube the EXTERNAL-IP stays `<pending>` because there is no cloud load balancer, so I used the NodePort URL. The logs show the regular `/health` calls from the Kubernetes probes.

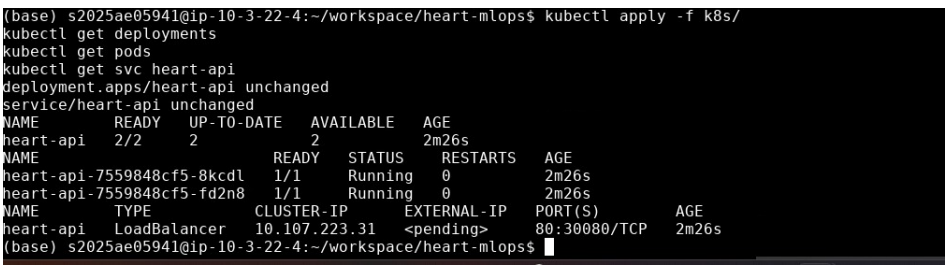


Figure 18: *Deployment (2/2), pods Running and the LoadBalancer service*

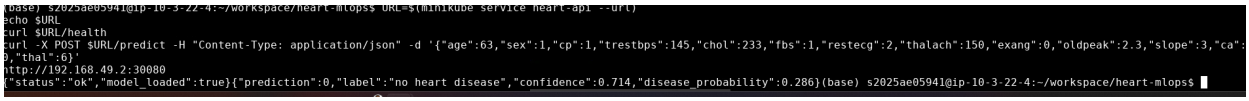


Figure 19: *Health check and prediction through the Kubernetes service*



Figure 20: *Pod logs showing readiness/liveness probe requests*

10. Monitoring and Logging

- **Logging:** a middleware in the API logs every request as `METHOD path -> status (time ms)`, and every prediction with its label and confidence. Logs are read with `docker logs` or `kubectl logs`.
- **Metrics:** the API exposes Prometheus metrics at `/metrics`: `api_requests_total` (by endpoint and status), `api_request_latency_seconds` (histogram) and `predictions_total` (by label).
- **Prometheus and Grafana** run as containers on the same Docker network as the API. Prometheus scrapes the API every 5 seconds, and Grafana is provisioned automatically with a data source and a 4-panel dashboard (requests/s, p95 latency, predictions by label, total requests and errors).

To test the monitoring, I used `src/generate_traffic.py` to send 300 random patients (plus a few invalid requests on purpose). The result was 288 successful predictions, 12 rejected requests (HTTP 422) and 31 health checks, i.e. 331 requests in total, and the same numbers appear in `/metrics` and in Grafana. The p95 latency was about 24 ms. The random traffic gave many more "heart disease" predictions (212 of 288) than the real data (46%). This shows how watching the prediction distribution can help to detect a change in the input data (data drift).

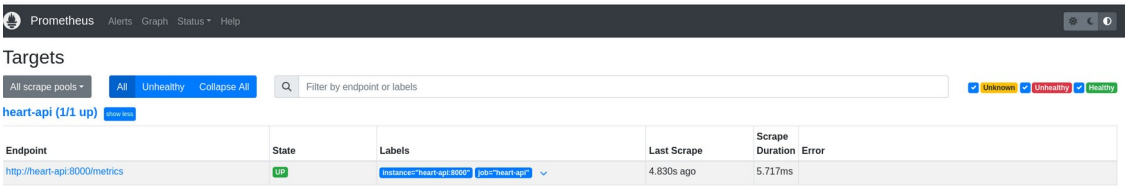


Figure 21: *Prometheus target heart-api is UP*

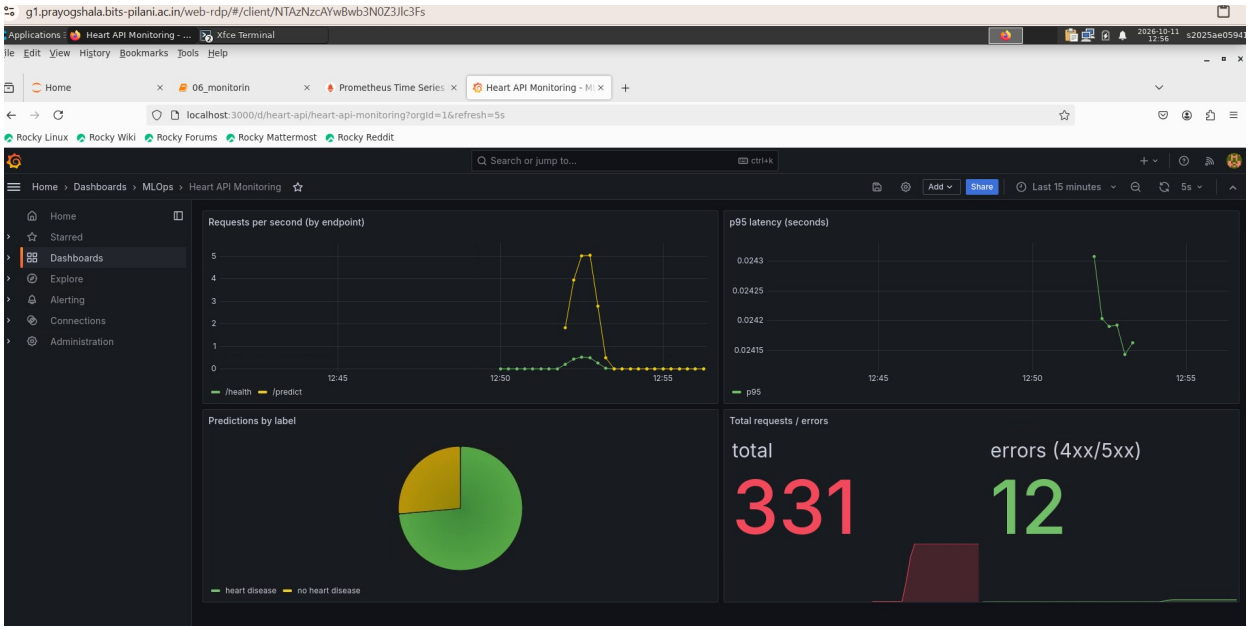


Figure 22: *Grafana dashboard after generating traffic*

```
.venv) s2025ae05941@ip-10-3-17-193:~/workspace/heart-mlops$ cd ~/workspace/heart-mlops
curl -s localhost:8000/metrics | grep -E "api_requests_total|predictions_total"
# HELP api_requests_total Total API requests
# TYPE api_requests_total counter
api_requests_total{endpoint="/health",status="200"} 31.0
api_requests_total{endpoint="/predict",status="422"} 12.0
api_requests_total{endpoint="/predict",status="200"} 288.0
# HELP predictions_total Predictions made, by label
# TYPE predictions_total counter
predictions_total{label="heart disease"} 212.0
predictions_total{label="no heart disease"} 76.0
(.venv) s2025ae05941@ip-10-3-17-193:~/workspace/heart-mlops$
```

Figure 23: *Raw Prometheus metrics from the /metrics endpoint*

```
(.venv) s2025ae05941@ip-10-3-17-193:~/workspace/heart-mlops$ docker logs heart-api --tail 15
INFO:     172.18.0.2:46372 - "GET /metrics HTTP/1.1" 200 OK
2026-10-11 07:27:19,311 INFO GET /metrics -> 200 (2.6 ms)
INFO:     172.18.0.2:46372 - "GET /metrics HTTP/1.1" 200 OK
2026-10-11 07:27:24,311 INFO GET /metrics -> 200 (2.8 ms)
INFO:     172.18.0.2:46372 - "GET /metrics HTTP/1.1" 200 OK
2026-10-11 07:27:29,311 INFO GET /metrics -> 200 (2.5 ms)
INFO:     172.18.0.2:46372 - "GET /metrics HTTP/1.1" 200 OK
2026-10-11 07:27:34,310 INFO GET /metrics -> 200 (2.6 ms)
INFO:     172.18.0.2:46372 - "GET /metrics HTTP/1.1" 200 OK
2026-10-11 07:27:39,310 INFO GET /metrics -> 200 (2.8 ms)
INFO:     172.18.0.2:46372 - "GET /metrics HTTP/1.1" 200 OK
2026-10-11 07:27:44,314 INFO GET /metrics -> 200 (2.7 ms)
INFO:     172.18.0.2:46372 - "GET /metrics HTTP/1.1" 200 OK
2026-10-11 07:27:49,310 INFO GET /metrics -> 200 (2.6 ms)
INFO:     172.18.0.2:46372 - "GET /metrics HTTP/1.1" 200 OK
(.venv) s2025ae05941@ip-10-3-17-193:~/workspace/heart-mlops$
```

Figure 24: *API request logs (docker logs)*

11. API Access Instructions

Table 5: *API endpoints*

Endpoint	Method	Description
/	GET	Welcome message
/health	GET	Health check (also used by the Kubernetes probes)
/predict	POST	Prediction for one patient (JSON with the 13 features)
/metrics	GET	Prometheus metrics
/docs	GET	Interactive Swagger UI

```
# Docker:      http://localhost:8000     Kubernetes:  URL=$(minikube service heart-api --url)
curl -X POST $URL/predict -H "Content-Type: application/json" \
  -d '{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,
       "thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}'

# Response
{"prediction":0,"label":"no heart disease","confidence":0.714,"disease_probability":0.286}
```

All setup steps (virtual environment, training, tests, Docker, Kubernetes and monitoring commands) are explained in the `README.md` of my repository.

12. Challenges and Solutions

Table 6: *Problems faced and how they were solved*

Challenge	Solution
Only `~/workspace` survives when the lab VM is terminated; Docker images, minikube and kubectl are lost	Kept the project and virtual environment in `~/workspace`; rebuilt the image and reinstalled kubectl on a new VM
Docker Hub rate limit (HTTP 429) when pulling the base image in CI	Used the official Python image from the AWS public ECR mirror
First CI run failed (flake8 W292, empty tests folder)	Fixed the files and added the tests; the failure also proved that the pipeline stops on errors
Stale directory listings on the NFS workspace	Opened a new terminal to see the latest files
EXTERNAL-IP stays <pending> on minikube	Accessed the LoadBalancer service through its NodePort URL

13. Conclusion

In this assignment I built a complete MLOps pipeline for heart-disease prediction. My selected Logistic Regression model reaches a cross-validated ROC-AUC of 0.909 and a test ROC-AUC of 0.959 with recall 0.929. The workflow is reproducible (pinned versions, fixed seeds, saved pipeline), tracked in MLflow, tested and trained automatically by GitHub Actions, packaged as a Docker image, deployed on Kubernetes with health probes and two replicas, and monitored with logs, Prometheus metrics and a Grafana dashboard. Through this work I learned how the different MLOps tools connect together into one workflow.

Future improvements: I would like to use the MLflow model registry with a database backend, push the Docker image to a registry from CI and deploy automatically (continuous deployment), add an Ingress and a HorizontalPodAutoscaler, add alerting rules in Prometheus, and monitor data drift by comparing live inputs with the training data.

References

- Detrano, R. et al. (1989). UCI Heart Disease Dataset, Cleveland Clinic Foundation. https://archive.ics.uci.edu/dataset/45/heart+disease
- scikit-learn documentation - https://scikit-learn.org
- MLflow documentation - https://mlflow.org/docs/latest
- FastAPI documentation - https://fastapi.tiangolo.com
- Kubernetes and minikube documentation - https://kubernetes.io/docs, https://minikube.sigs.k8s.io
- Prometheus and Grafana documentation - https://prometheus.io/docs, https://grafana.com/docs